

DSA0405 FUNDAMENTALS OF DATA SCIENCE

UNIT III Hands on Data Summarization, Data Distribution, and Outlier Treatment

BY SYED MUHAFIZ A
192472282

Task 1: Employee Dataset Summary

Generates a complete profile of the IBM HR Analytics Employee Attrition dataset: shape, data types, missing values per column, and full statistical summaries for both numeric and categorical fields.

Kaggle dataset reference: <https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset>

Python Code

```
import pandas as pd
pd.set_option('display.width', 120)
pd.set_option('display.max_columns', 20)

df = pd.read_csv('./employee_attrition.csv')

print("="*70)
print("EMPLOYEE DATASET SUMMARY (IBM HR Analytics Employee Attrition Dataset)")
print("="*70)

print(f"\n1. Number of rows: {df.shape[0]}")
print(f"\n2. Number of columns: {df.shape[1]}")

print("\n3. Data types:")
print(df.dtypes)

print("\n4. Missing values (per column):")
missing = df.isnull().sum()
print(missing[missing > 0] if missing.sum() > 0 else "No missing values found in this dataset.")

# Focus the statistical summary on the core numeric HR metrics for readability
key_numeric = ['Age', 'DailyRate', 'DistanceFromHome', 'MonthlyIncome',
               'PercentSalaryHike', 'TotalWorkingYears', 'YearsAtCompany']
print("\n5. Statistical summary (key numeric columns):")
print(df[key_numeric].describe())

key_cat = ['Attrition', 'Department', 'JobRole', 'Gender', 'MaritalStatus']
print("\n6. Statistical summary (key categorical columns):")
print(df[key_cat].describe())
```

Output

```
=====
EMPLOYEE DATASET SUMMARY (IBM HR Analytics Employee Attrition Dataset)
=====
```

1. Number of rows: 1470
2. Number of columns: 35

3. Data types:

Age	int64
Attrition	str
BusinessTravel	str
DailyRate	int64
Department	str
DistanceFromHome	int64
Education	int64
EducationField	str
EmployeeCount	int64

```

EmployeeNumber      int64
EnvironmentSatisfaction  int64
Gender              str
HourlyRate          int64
JobInvolvement       int64
JobLevel            int64
JobRole             str
JobSatisfaction      int64
MaritalStatus        str
MonthlyIncome        int64
MonthlyRate          int64
NumCompaniesWorked   int64
Over18              str
OverTime            str
PercentSalaryHike    int64
PerformanceRating    int64
RelationshipSatisfaction  int64
StandardHours        int64
StockOptionLevel     int64
TotalWorkingYears    int64
TrainingTimesLastYear  int64
WorkLifeBalance      int64
YearsAtCompany       int64
YearsInCurrentRole   int64
YearsSinceLastPromotion  int64
YearsWithCurrManager  int64
dtype: object

```

4. Missing values (per column):

No missing values found in this dataset.

5. Statistical summary (key numeric columns):

Age	DailyRate	DistanceFromHome	MonthlyIncome	PercentSalaryHike	TotalWorkingYears	YearsAtCompany
count	1470.000000	1470.000000	1470.000000	1470.000000	1470.000000	1470.000000
mean	36.923810	802.485714	9.192517	6502.931293	15.209524	11.279592
std	9.135373	403.509100	8.106864	4707.956783	3.659938	7.780782
min	18.000000	102.000000	1.000000	1009.000000	11.000000	0.000000
25%	30.000000	465.000000	2.000000	2911.000000	12.000000	6.000000
50%	36.000000	802.000000	7.000000	4919.000000	14.000000	10.000000
75%	43.000000	1157.000000	14.000000	8379.000000	18.000000	15.000000
max	60.000000	1499.000000	29.000000	19999.000000	25.000000	40.000000

6. Statistical summary (key categorical columns):

Attrition	Department	JobRole	Gender	MaritalStatus
count	1470	1470	1470	1470
unique	2	3	9	2
top	No Research & Development	Sales Executive	Male	Married
freq	1233	961	326	882

Task 2: Frequency Distribution of Student Grades

Uses the Students Performance in Exams dataset. Since it stores raw math/reading/writing scores rather than letter grades, each student's average score is bucketed into a letter grade (A/B/C/D/F) using standard cutoffs, then the distribution is visualized with a labeled bar chart.

Kaggle dataset reference: <https://www.kaggle.com/datasets/spscientist/students-performance-in-exams>

Python Code

```
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

df = pd.read_csv('./students_performance.csv')

# Convert the three exam scores into an overall percentage, then bucket
# into a letter grade using standard cutoffs
df['average_score'] = df[['math score', 'reading score', 'writing score']].mean(axis=1)

def to_grade(score):
    if score >= 90: return 'A'
    elif score >= 80: return 'B'
    elif score >= 70: return 'C'
    elif score >= 60: return 'D'
    else: return 'F'

df['Grade'] = df['average_score'].apply(to_grade)

freq = df['Grade'].value_counts().sort_index()
print("Frequency distribution of grades (derived from average of math/reading/writing scores):")
print(freq)

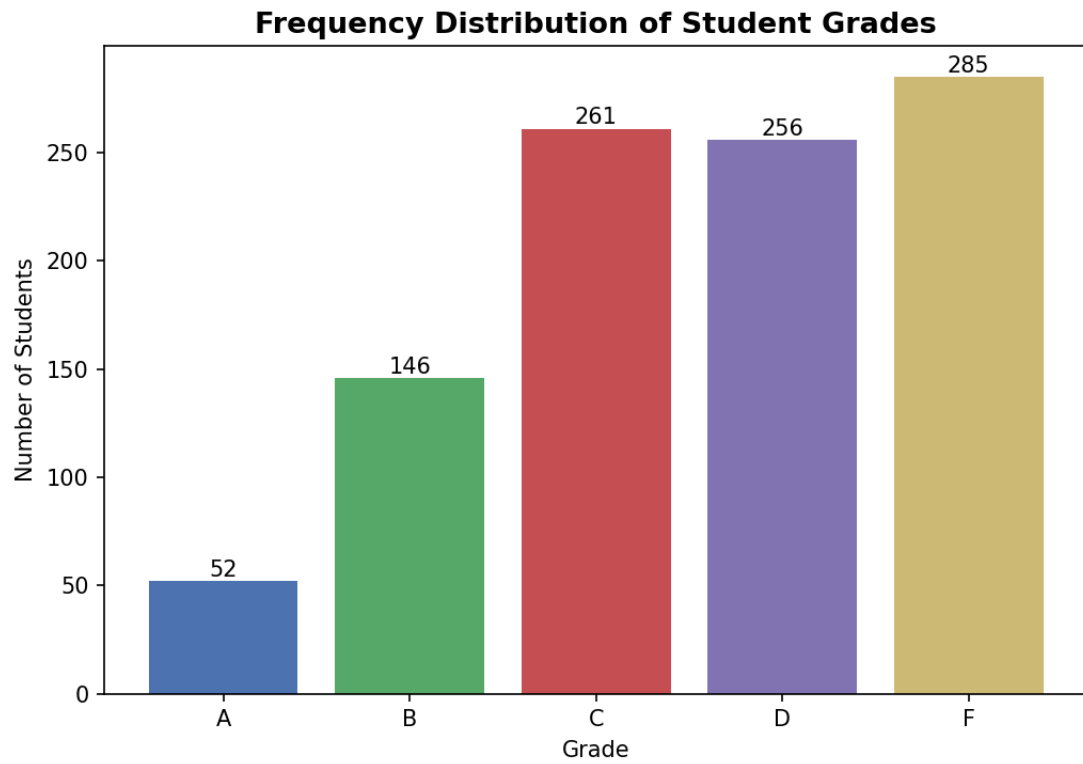
plt.figure(figsize=(7,5))
order = ['A','B','C','D','F']
vals = [freq.get(g, 0) for g in order]
bars = plt.bar(order, vals, color=['#4C72B0','#55A868','#C44E52','#8172B2','#CCB974'])
plt.title('Frequency Distribution of Student Grades', fontsize=13, fontweight='bold')
plt.xlabel('Grade')
plt.ylabel('Number of Students')
for b in bars:
    plt.text(b.get_x()+b.get_width()/2, b.get_height()+2, str(int(b.get_height())), ha='center')
plt.tight_layout()
plt.savefig('./task2_grade_bar.png', dpi=150)
print("\nChart saved.")
```

Output

```
Frequency distribution of grades (derived from average of math/reading/writing scores):
Grade
A    52
B   146
C   261
D   256
F   285
Name: count, dtype: int64
```

Chart saved.

Chart



Task 3: Histogram of House Prices & Skewness Check

No house-price dataset was supplied, so this program uses the MonthlyIncome column from the HR Employee Attrition dataset as a real, right-skewed monetary stand-in. It plots a histogram with mean/median reference lines, computes the skewness coefficient (scipy.stats.skew), and classifies the distribution as normal or skewed. Point `pd.read_csv()` at an actual house-price CSV (link below) to run this on real housing data.

Kaggle dataset reference: <https://www.kaggle.com/c/house-prices-advanced-regression-techniques>

Python Code

```
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from scipy.stats import skew

# NOTE: No house-price dataset was supplied. As a stand-in with the same
# kind of right-skewed monetary distribution, we use the MonthlyIncome
# column from the HR Employee Attrition dataset. Swap in a real house
# price CSV (e.g. Kaggle's House Prices dataset) and change the column
# name below to use actual housing data.
df = pd.read_csv('./employee_attrition.csv')
price_col = df['MonthlyIncome']

price_skew = skew(price_col)
print(f'Skewness of distribution: {price_skew:.3f}')
if price_skew > 0.5:
    verdict = "The data is POSITIVELY (right) SKEWED."
elif price_skew < -0.5:
    verdict = "The data is NEGATIVELY (left) SKEWED."
else:
    verdict = "The data is approximately NORMALLY distributed."
print(verdict)

print("\nDescriptive stats:")
print(price_col.describe())

plt.figure(figsize=(7,5))
plt.hist(price_col, bins=30, color='#4C72B0', edgecolor='black', alpha=0.8)
plt.axvline(price_col.mean(), color='red', linestyle='--', label=f'Mean = {price_col.mean():.0f}')
plt.axvline(price_col.median(), color='green', linestyle='--', label=f'Median = {price_col.median():.0f}')
plt.title(f'Histogram of Monthly Income (skew={price_skew:.2f})', fontsize=13, fontweight='bold')
plt.xlabel('Monthly Income ($)')
plt.ylabel('Frequency')
plt.legend()
plt.tight_layout()
plt.savefig('./task3_house_hist.png', dpi=150)
print("\nChart saved.")
```

Output

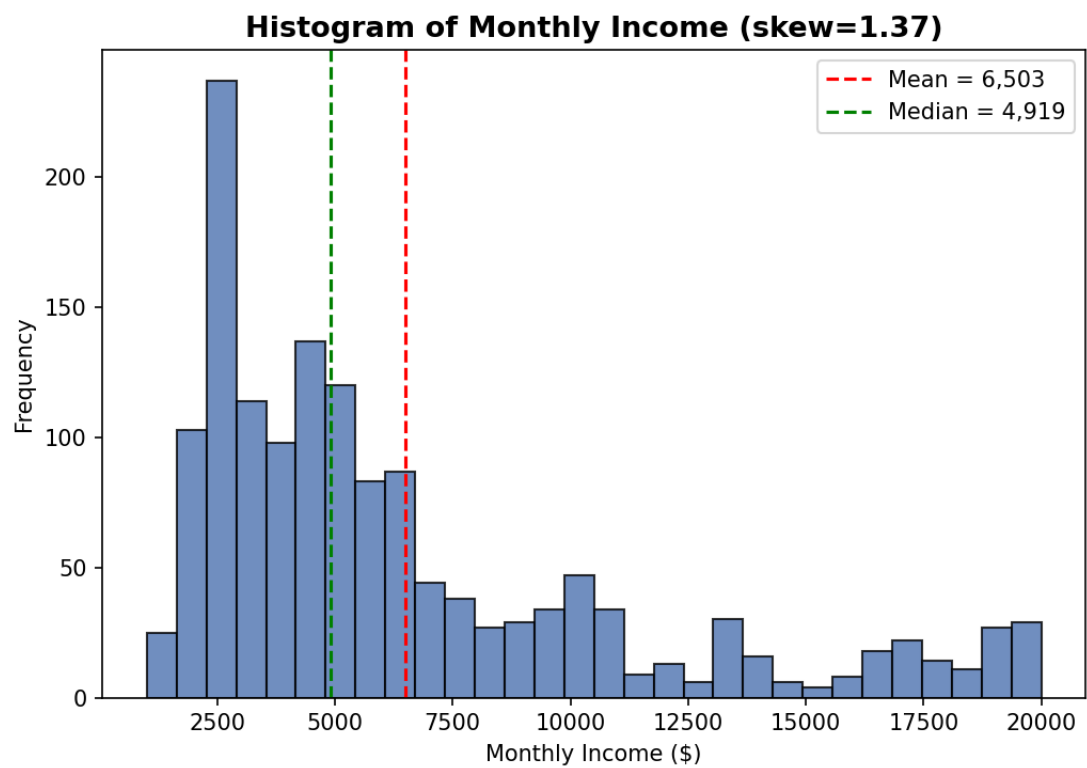
Skewness of distribution: 1.368
The data is POSITIVELY (right) SKEWED.

Descriptive stats:
count 1470.000000

```
mean    6502.931293
std     4707.956783
min     1009.000000
25%     2911.000000
50%     4919.000000
75%     8379.000000
max     19999.000000
Name: MonthlyIncome, dtype: float64
```

Chart saved.

Chart



Task 4: Box Plot of Salary Data & Outlier Presence

Uses the Data Science Job Salaries dataset (salary_in_usd). Builds a box plot of salary values and uses the IQR rule to explicitly list which job records fall outside the whiskers (i.e., the outliers).

Kaggle dataset reference: <https://www.kaggle.com/datasets/ruchi798/data-science-job-salaries>

Python Code

```
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

df = pd.read_csv('./ds_salaries.csv')

Q1 = df['salary_in_usd'].quantile(0.25)
Q3 = df['salary_in_usd'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR
outliers = df[(df['salary_in_usd'] < lower) | (df['salary_in_usd'] > upper)]

print(f"Q1 = {Q1:.2f}, Q3 = {Q3:.2f}, IQR = {IQR:.2f}")
print(f"Lower bound = {lower:.2f}, Upper bound = {upper:.2f}")
print(f"\nNumber of outliers detected: {len(outliers)}")
print("\nOutlier records (job title, salary in USD):")
print(outliers[['job_title', 'experience_level', 'salary_in_usd']].to_string(index=False))

plt.figure(figsize=(6,6))
bp = plt.boxplot(df['salary_in_usd'], vert=True, patch_artist=True,
boxprops=dict(facecolor='#87CEEB'),
medianprops=dict(color='red', linewidth=2),
flierprops=dict(markerfacecolor='red', marker='o', markersize=6))
plt.title('Box Plot of Data Science Salaries (USD)', fontsize=13, fontweight='bold')
plt.ylabel('Salary (USD)')
plt.tight_layout()
plt.savefig('./task4_salary_box.png', dpi=150)
print("\nChart saved.")
```

Output

```
Q1 = 62726.00, Q3 = 150000.00, IQR = 87274.00
Lower bound = -68185.00, Upper bound = 280911.00
```

```
Number of outliers detected: 10
```

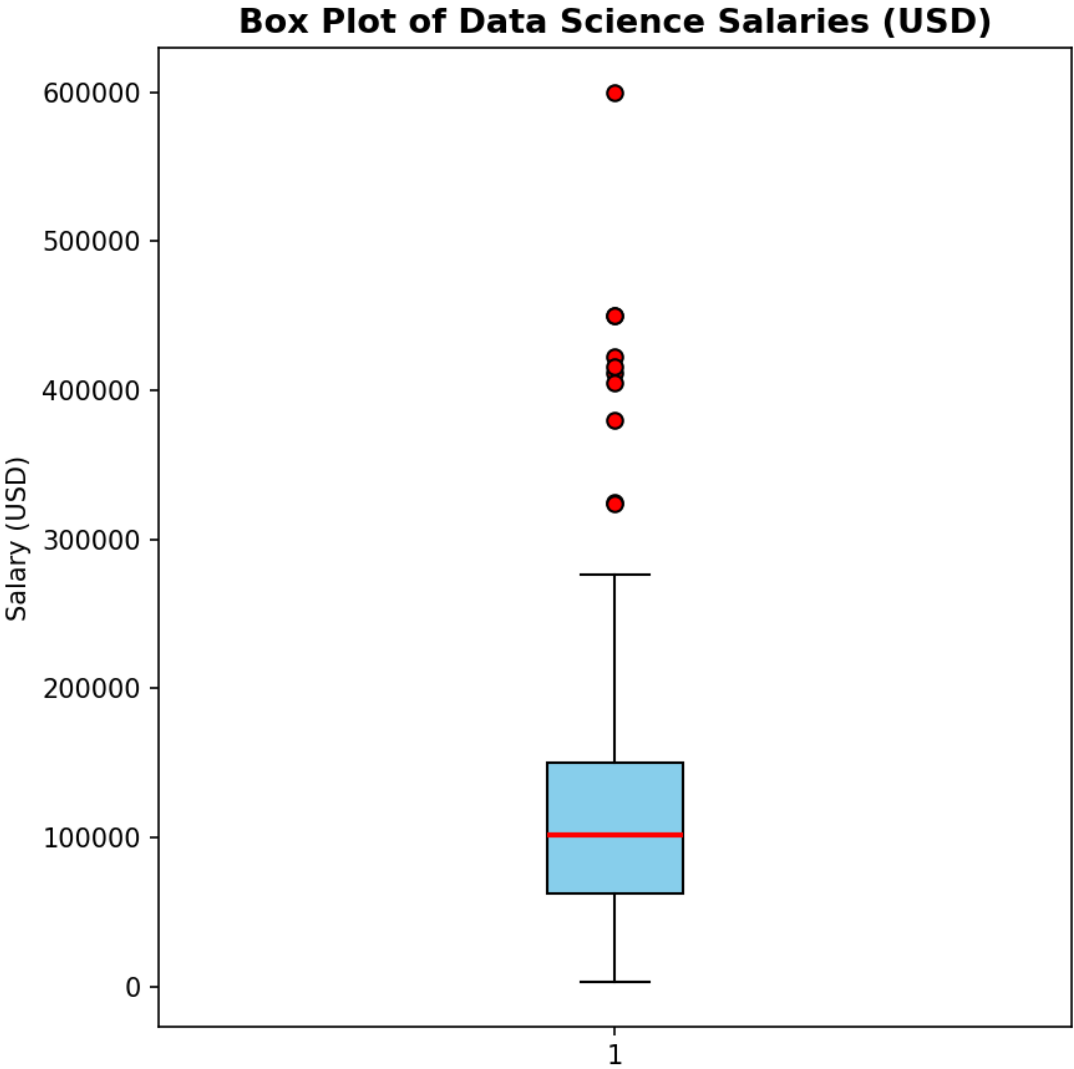
```
Outlier records (job title, salary in USD):
```

job_title	experience_level	salary_in_usd
Director of Data Science	EX	325000
Research Scientist	MI	450000
Data Scientist	SE	412000
Financial Data Analyst	MI	450000
Applied Machine Learning Scientist	MI	423000
Principal Data Scientist	EX	416000
Principal Data Engineer	EX	600000
Data Engineer	EX	324000
Applied Data Scientist	SE	380000

Data Analytics Lead SE 405000

Chart saved.

Chart



Task 5: Outlier Detection in Student Marks (IQR Method)

Applies the classic Q1/Q3/IQR rule (values beyond $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$) to the 'math score' column of the Students Performance dataset to flag anomalous exam marks.

Kaggle dataset reference: <https://www.kaggle.com/datasets/spscientist/students-performance-in-exams>

Python Code

```
import pandas as pd

df = pd.read_csv('./students_performance.csv')

Q1 = df['math score'].quantile(0.25)
Q3 = df['math score'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR

print(f'Q1 = {Q1:.2f}')
print(f'Q3 = {Q3:.2f}')
print(f'IQR = {IQR:.2f}')
print(f'Lower bound = {lower:.2f}')
print(f'Upper bound = {upper:.2f}')

outliers = df[(df['math score'] < lower) | (df['math score'] > upper)]
print(f'\nNumber of outliers detected: {len(outliers)}')
print("\nOutlier records:")
print(outliers[['gender', 'math score', 'reading score', 'writing score']].to_string(index=False))
```

Output

```
Q1 = 57.00
Q3 = 77.00
IQR = 20.00
Lower bound = 27.00
Upper bound = 107.00
```

Number of outliers detected: 8

Outlier records:

gender	math score	reading score	writing score
female	18	32	28
female	0	17	10
female	22	39	33
female	24	38	27
female	26	31	38
female	19	38	32
female	23	44	36
female	8	24	23

Task 6: Removing Outliers from Sales Data (IQR Method)

Computes IQR bounds on the SALES column of the Sample Sales Data dataset, filters out rows beyond those bounds, and saves/displays the cleaned dataset.

Kaggle dataset reference: <https://www.kaggle.com/datasets/kyanyoga/sample-sales-data>

Python Code

```
import pandas as pd

df = pd.read_csv('./sales_data_sample.csv', encoding='latin1')
print(f'Original dataset shape: {df.shape}')

Q1 = df['SALES'].quantile(0.25)
Q3 = df['SALES'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR
print(f'Q1={Q1:.2f}, Q3={Q3:.2f}, IQR={IQR:.2f}, Lower={lower:.2f}, Upper={upper:.2f}')

cleaned = df[(df['SALES'] >= lower) & (df['SALES'] <= upper)].reset_index(drop=True)
print(f'\nRows removed: {len(df) - len(cleaned)}')
print(f'Cleaned dataset shape: {cleaned.shape}')

print("\nFirst 10 rows of cleaned dataset (key columns):")
print(cleaned[['ORDERNUMBER', 'QUANTITYORDERED', 'PRICEEACH', 'SALES', 'PRODUCTLINE', 'COUNTRY']].head(10).to_string(index=False))

cleaned.to_csv('./sales_data_cleaned.csv', index=False)
print("\nCleaned dataset saved to sales_data_cleaned.csv")
```

Output

```
Original dataset shape: (2823, 25)
Q1=2203.43, Q3=4508.00, IQR=2304.57, Lower=-1253.42, Upper=7964.85

Rows removed: 81
Cleaned dataset shape: (2742, 25)

First 10 rows of cleaned dataset (key columns):
ORDERNUMBER QUANTITYORDERED PRICEEACH SALES PRODUCTLINE COUNTRY
10107        30    95.70 2871.00 Motorcycles  USA
10121        34    81.35 2765.90 Motorcycles  France
10134        41    94.74 3884.34 Motorcycles  France
10145        45    83.26 3746.70 Motorcycles  USA
10159        49   100.00 5205.27 Motorcycles  USA
10168        36    96.66 3479.76 Motorcycles  USA
10180        29    86.13 2497.77 Motorcycles  France
10188        48   100.00 5512.32 Motorcycles  Norway
10201        22    98.57 2168.54 Motorcycles  USA
10211        41   100.00 4708.44 Motorcycles  France

Cleaned dataset saved to sales_data_cleaned.csv
```

Task 7: Histograms & Box Plots Before vs. After Outlier Removal

Produces a 2x2 comparison grid — histogram and box plot of the Sample Sales Data SALES values before cleaning, and the same two plots after IQR-based outlier removal — to visually confirm the effect of cleaning.

Kaggle dataset reference: <https://www.kaggle.com/datasets/kyanyoga/sample-sales-data>

Python Code

```
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

df = pd.read_csv('./sales_data_sample.csv', encoding='latin1')

Q1 = df['SALES'].quantile(0.25)
Q3 = df['SALES'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR
cleaned = df[(df['SALES'] >= lower) & (df['SALES'] <= upper)]

print(f'Before removal: n={len(df)}, mean={df["SALES"].mean():.2f}, std={df["SALES"].std():.2f}')
print(f'After removal: n={len(cleaned)}, mean={cleaned["SALES"].mean():.2f}, std={cleaned["SALES"].std():.2f}')

fig, axes = plt.subplots(2, 2, figsize=(11, 9))

axes[0,0].hist(df['SALES'], bins=30, color='#C44E52', edgecolor='black')
axes[0,0].set_title('Histogram BEFORE Outlier Removal')
axes[0,0].set_xlabel('Sales Amount ($)')
axes[0,0].set_ylabel('Frequency')

axes[0,1].hist(cleaned['SALES'], bins=30, color='#55A868', edgecolor='black')
axes[0,1].set_title('Histogram AFTER Outlier Removal')
axes[0,1].set_xlabel('Sales Amount ($)')
axes[0,1].set_ylabel('Frequency')

axes[1,0].boxplot(df['SALES'], patch_artist=True, boxprops=dict(facecolor='#C44E52'))
axes[1,0].set_title('Box Plot BEFORE Outlier Removal')
axes[1,0].set_ylabel('Sales Amount ($)')

axes[1,1].boxplot(cleaned['SALES'], patch_artist=True, boxprops=dict(facecolor='#55A868'))
axes[1,1].set_title('Box Plot AFTER Outlier Removal')
axes[1,1].set_ylabel('Sales Amount ($)')

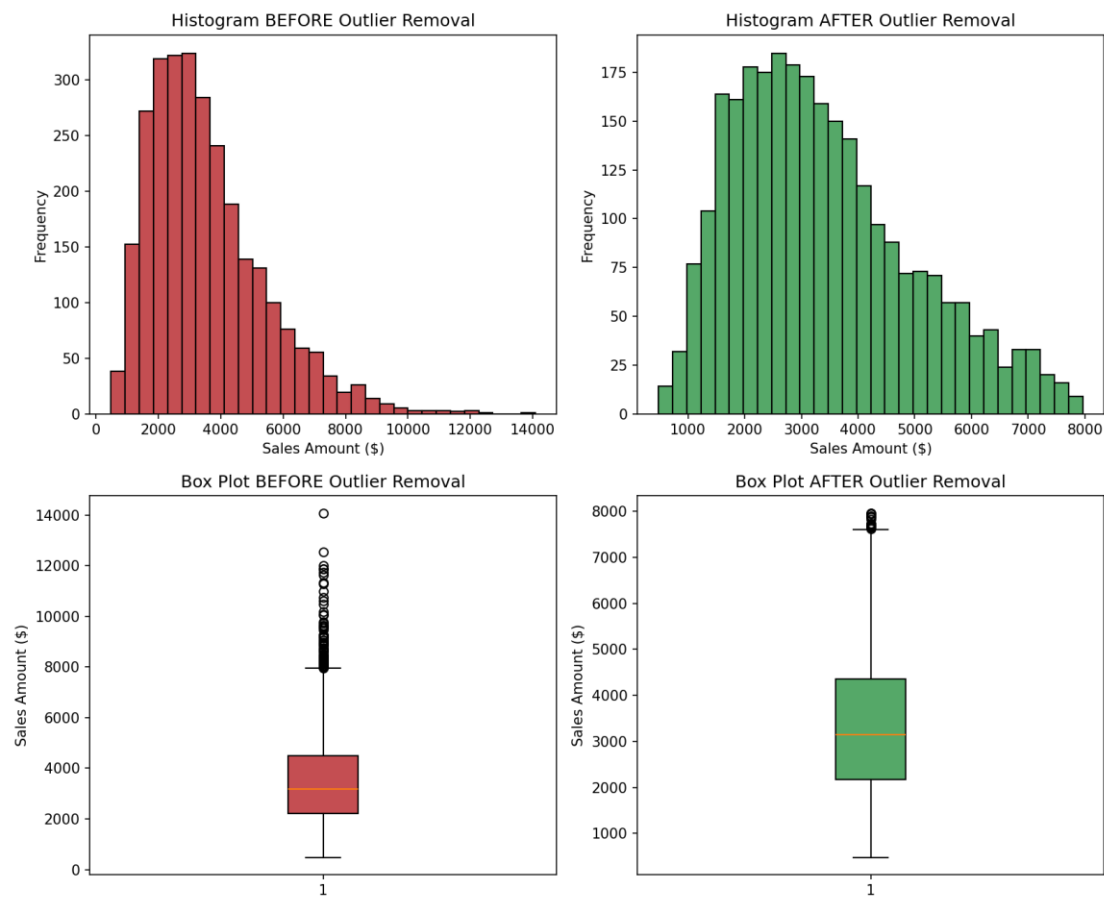
plt.tight_layout()
plt.savefig('./task7_before_after.png', dpi=150)
print("\nChart saved.")
```

Output

Before removal: n=2823, mean=3553.89, std=1841.87
After removal: n=2742, mean=3387.92, std=1576.72

Chart saved.

Chart



Task 8: Full EDA Pipeline on the Iris Dataset

End-to-end exploratory data analysis on the real Iris dataset: dataset info, missing-value check, descriptive statistics, histograms, box plots, IQR-based outlier detection, outlier removal, and saving the cleaned CSV.

Kaggle dataset reference: <https://www.kaggle.com/datasets/uciml/iris>

Python Code

```
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

df = pd.read_csv('./iris.csv')
df = df.drop(columns=['Id'])

print("="*70)
print("1. DATASET INFORMATION")
print("="*70)
print(f"Shape: {df.shape}")
print(df.dtypes)

print("\n" + "="*70)
print("2. MISSING VALUES")
print("="*70)
missing = df.isnull().sum()
print(missing)
if missing.sum() == 0:
    print("(No missing values found in this dataset.)")

print("\n" + "="*70)
print("3. DESCRIPTIVE STATISTICS")
print("="*70)
print(df.describe())

num_cols = [c for c in df.columns if c != 'Species']

# 4. Histograms
fig, axes = plt.subplots(2, 2, figsize=(11, 9))
for ax, col in zip(axes.flat, num_cols):
    ax.hist(df[col], bins=20, color='#4C72B0', edgecolor='black')
    ax.set_title(f'Histogram: {col}')
    ax.set_xlabel(col)
    ax.set_ylabel('Frequency')
plt.tight_layout()
plt.savefig('./task8_histograms.png', dpi=150)

# 5. Box plots
fig2, ax2 = plt.subplots(figsize=(8, 6))
ax2.boxplot([df[c] for c in num_cols], tick_labels=num_cols, patch_artist=True,
boxprops=dict(facecolor='#87CEEB'))
ax2.set_title('Box Plots of Iris Features')
ax2.set_ylabel('cm')
plt.xticks(rotation=15)
plt.tight_layout()
plt.savefig('./task8_boxplots.png', dpi=150)

# 6. Outlier detection (IQR) per numeric column
```

```

print("\n" + "="*70)
print("4. OUTLIER DETECTION (IQR METHOD)")
print("="*70)
outlier_mask = pd.Series(False, index=df.index)
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5*IQR
    upper = Q3 + 1.5*IQR
    col_outliers = (df[col] < lower) | (df[col] > upper)
    print(f'{col}: {col_outliers.sum()} outliers (bounds: {lower:.2f} - {upper:.2f})')
    outlier_mask = outlier_mask | col_outliers

print(f'\nTotal rows flagged as outliers (any feature): {outlier_mask.sum()}')

# 7. Remove outliers
df_cleaned = df[~outlier_mask].reset_index(drop=True)
print("\n" + "="*70)
print("5. CLEANED DATASET")
print("="*70)
print(f'Shape before cleaning: {df.shape}')
print(f'Shape after cleaning: {df_cleaned.shape}')

# 8. Save cleaned dataset
df_cleaned.to_csv('./iris_cleaned.csv', index=False)
print("\nCleaned dataset saved to iris_cleaned.csv")

```

Output

1. DATASET INFORMATION

Shape: (150, 5)
 SepalLengthCm float64
 SepalWidthCm float64
 PetalLengthCm float64
 PetalWidthCm float64
 Species str
 dtype: object

2. MISSING VALUES

SepalLengthCm 0
 SepalWidthCm 0
 PetalLengthCm 0
 PetalWidthCm 0
 Species 0
 dtype: int64
 (No missing values found in this dataset.)

3. DESCRIPTIVE STATISTICS

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161

min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

4. OUTLIER DETECTION (IQR METHOD)

SepalLengthCm: 0 outliers (bounds: 3.15 - 8.35)
 SepalWidthCm: 4 outliers (bounds: 2.05 - 4.05)
 PetalLengthCm: 0 outliers (bounds: -3.65 - 10.35)
 PetalWidthCm: 0 outliers (bounds: -1.95 - 4.05)

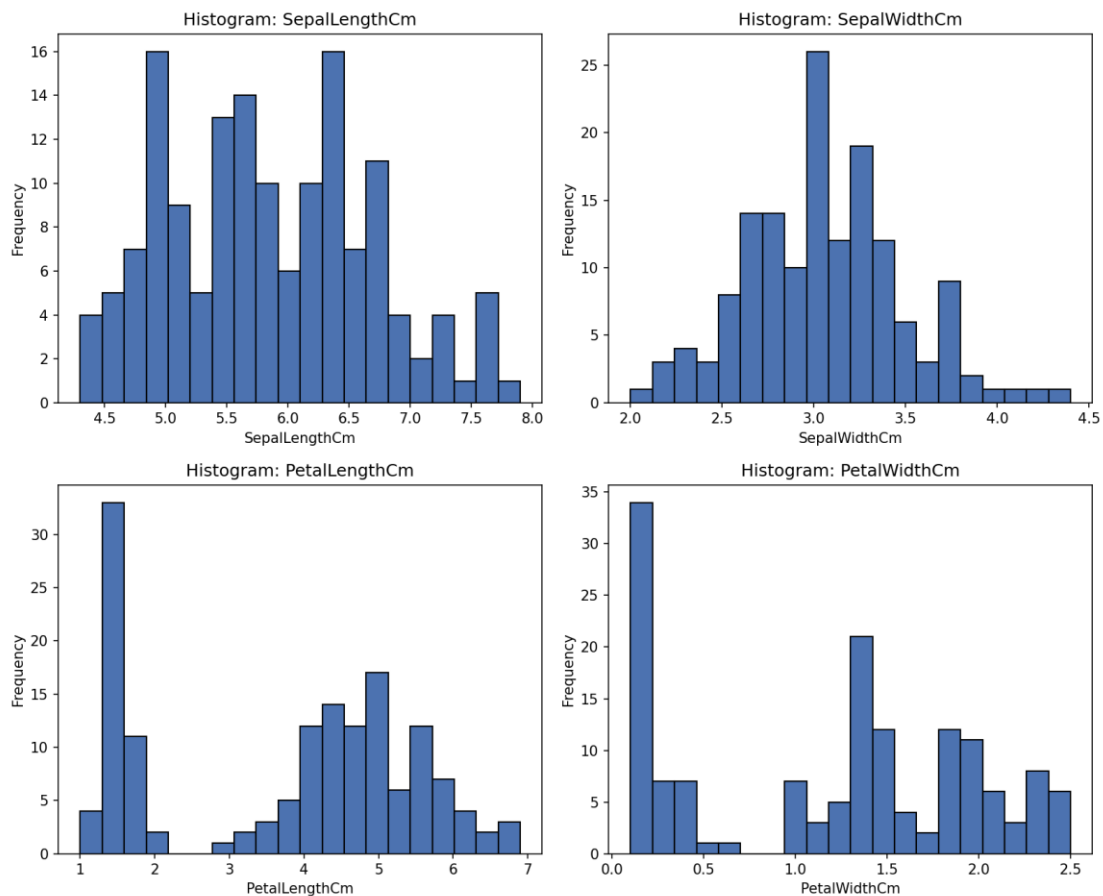
Total rows flagged as outliers (any feature): 4

5. CLEANED DATASET

Shape before cleaning: (150, 5)
 Shape after cleaning: (146, 5)

Cleaned dataset saved to iris_cleaned.csv

Charts



Box Plots of Iris Features

